
LOOPGYM: TRAINING FOR ROLLOUT-COMBINE-FILTER INFERENCE IN TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Loop-invariant generation is compositional: one response may contain a useful bound, another a relational equality, and neither need prove the target alone. This makes pass@1 a poor training and evaluation objective for an inference procedure that can combine partial answers. We study this problem under *target-hidden loop verification*, where the model sees the precondition, guard, and body but not the assertion or postcondition Q . Our inference procedure samples multiple responses, splits conjunctive invariants into clauses, unions them, iteratively removes syntax failures, and uses verifier-checked Houdini pruning to retain an inductive subset before Q is restored for the final proof. We align both stages of training with this rollout-combine-filter procedure. First, synthetic supervised targets are formed by applying it to multiple target-hidden responses and serializing the combined verified set, thereby compressing a multi-response search result into one response. Second, reinforcement learning replaces an independent verifier reward with credit assignment at two levels. Within a rollout, the verifier projects its clauses onto a sound subset, and we reward the strength of that subset rather than whole-response correctness. Across rollouts, a joint group reward credits what each subset contributes beyond its peers. The reward uses reachable executions and conservatively filtered synthetic negative examples, never Q . The resulting objective is not to maximize standalone proof rate, but to train responses that combine into a stronger verified set under a fixed inference budget. On 832 C programs, five-rollout inference verifies 74.76% of tasks, compared with 61.78% for the strongest external baseline. Reinforcement learning raises combine@10 from 52.40% to 56.97% even as pass@1 decreases, and reduces the rollouts needed to reach 95% of the measured maximum from 30 to 10. Thus ten trained-model responses outperform the combination of one hundred base-model responses.

1 INTRODUCTION

Verifying any program with a loop requires finding an adequate invariant: no proof can otherwise cover arbitrarily many iterations, and writing invariants by hand remains a central bottleneck to verifying real software. Loop invariants summarize an unbounded computation in a finite assertion. They must be inductive while retaining enough program knowledge to establish the desired property at loop exit. A verifier gives a sharp test of soundness, but not a graded measure of precision: *true* can be inductive while proving nothing useful. Moreover, the exact reachable-state set may be expensive to compute or inexpressible in the annotation language. The learning problem is therefore not merely to produce a sound invariant, but to approach a stronger sound over-approximation. We formalize soundness, logical strength, and target sufficiency in Section 2.

We study this alignment problem under *target-hidden loop verification*, where the target is withheld until after the invariant set has been fixed (Section 2). Existing methods can use the target, failed obligations, or counterexamples to guide search (Gulwani et al., 2008; Gupta & Rybalchenko, 2009; Dillig et al., 2013; Alur et al., 2013; Garg et al., 2014; 2016; Si et al., 2020; Kamath et al., 2023; Cao et al., 2025). Hiding Q prevents copying an inductive postcondition. Final success is unchanged: the verifier must prove soundness and sufficiency for the restored target. This rules out direct target sufficiency as a training reward and makes a dense, target-independent strength signal necessary.

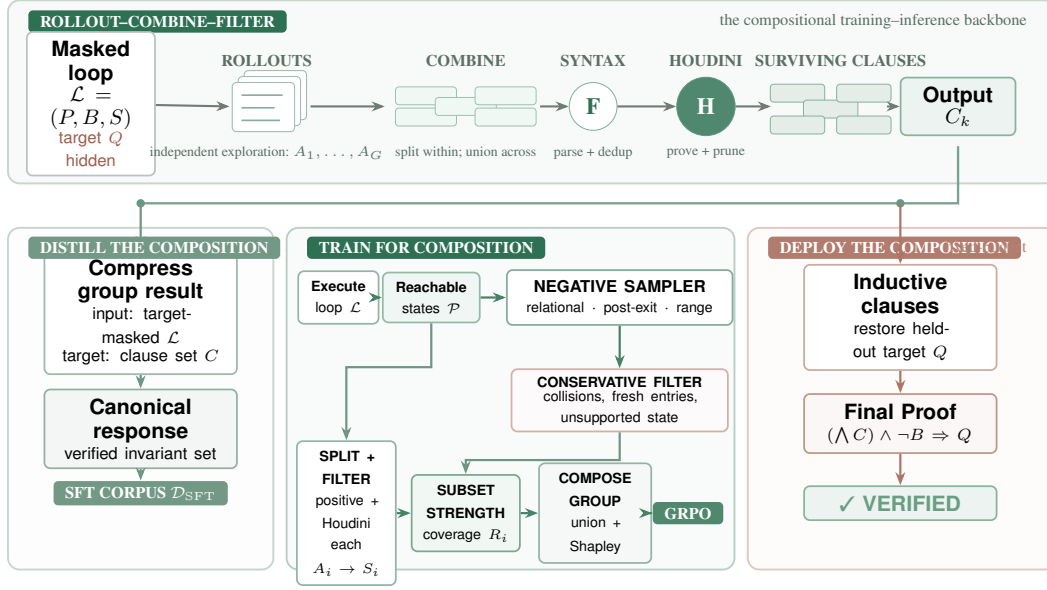


Figure 1: **Rollout-combine-filter is the organizing principle.** Multiple target-hidden rollouts explore independently; their clauses are split within responses, unioned across responses, and filtered into one combined invariant. SFT distills this composition. RL mirrors it by rewarding the sound subset inside each rollout and its complementary contribution across the group; during training, reachable positive examples pre-filter clauses before Houdini. Deployment restores Q only after the same composition is fixed.

The same task also offers a useful structure: sound invariants compose. The conjunction of two inductive predicates is inductive and no weaker than either predicate. One response may provide a bound and another a relational equality, so their union can prove a target that neither proves alone. We therefore treat the filtered composition of a fixed rollout group as the deployed answer. Our objective is $\text{combine}@k$, rather than $\text{pass}@1$ or $\text{pass}@k$: with as few responses as possible, the policy should discover a clause pool whose union survives Houdini and proves the restored target. This compositionality is the organizing principle of our method: every other component is designed to make rollout-combine-filter effective under a finite sampling budget.

This objective exposes a training-inference mismatch. An independent verifier reward assigns the same score to a rollout regardless of what its peers already discovered. It cannot distinguish a new useful relation from a redundant copy, and can collapse repeated sampling onto the same easy high-reward clauses. A whole-response pass/fail reward also discards partial progress inside an imperfect response: one invalid clause can obscure several useful ones.

We address credit assignment along both axes. Within each rollout, we split conjunctions into clauses and use the verifier as a projection operator: syntax filtering and Houdini retain a sound subset, but provide no extra reward merely because the original response is syntactically valid or every proposed clause survives. We reward the behavioral strength of the retained subset, so useful clauses receive partial credit even when their response is imperfect. Across rollouts, a closed-form Shapley allocation credits each subset for negative coverage not already supplied by its peers. The resulting reward is therefore both clause-decomposed within a response and group-conditioned across responses.

LOOPGYM implements this idea with one rollout-combine-filter operator shared by training and deployment (Section 4). SFT distills a filtered multi-response union into one stronger response. GRPO supplies groups of rollouts, and our group-conditioned reward trains each response for its contribution to their composition. At inference, the same clause splitting, union, syntax filtering, and Houdini pruning produce the final invariant before Q is restored.

Our experiments deliberately separate standalone success from combined inference. On 832 integer C programs, five target-hidden responses with union and Houdini verify 622 programs (74.76%), compared with 514 (61.78%) for the strongest external baseline. On Qwen3-8B, reinforcement

learning decreases direct pass@1 from 9.30% to 4.88% but increases combine@10 from 52.40% to 56.97%. Ten responses from the trained policy exceed combine@100 of the base model, reducing the rollouts needed to reach 95% of the measured maximum from 30 to 10. Training thus compresses broad base-model exploration into a much smaller deployment budget rather than optimizing one standalone target-proving answer.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification around rollout–combine–filter: the deployed and optimized object is the verified composition of a fixed response group, measured by combine@ k , rather than a standalone response measured by pass@1.
- We align training credit with that compositional object. Synthetic SFT targets distill filtered combinations, while RL decomposes credit over the sound clauses within each rollout and composes credit across rollouts according to their complementary contribution, without observing Q .
- We show that this alignment improves both combined verification and rollout efficiency, and audit the synthetic negative examples that support the reward.

2 PRELIMINARIES

2.1 LOOP VERIFICATION

A verification task consists of a loop $\mathcal{L} = (P, B, S)$ and a postcondition Q , where P is the precondition, B is the loop guard, and S is the loop body. Let $\text{Reach}(\mathcal{L})$ denote the states reachable at the loop head, including the final guard test at loop exit.

An invariant I is *sound* (equivalently, inductive) when it is established by P and preserved by one iteration:

$$P \Rightarrow I, \quad \{I \wedge B\} S \{I\}.$$

These obligations ensure that I holds on every state in $\text{Reach}(\mathcal{L})$ (Floyd, 1967; Hoare, 1969).

We say that I_1 is *stronger* than I_2 , written $I_1 \succeq I_2$, when

$$\models I_1 \Rightarrow I_2.$$

An invariant I^* is *exact* when

$$\llbracket I^* \rrbracket = \text{Reach}(\mathcal{L}).$$

Because every sound invariant contains all reachable states, an exact invariant is stronger than every sound invariant of the same loop:

$$I^* \text{ exact, } J \text{ sound} \implies \models I^* \Rightarrow J.$$

Exactness is a semantic ideal; the reachable states need not have a finite representation in the annotation fragment used by the generator.

Finally, a sound invariant I is *Q-sufficient* when it proves the postcondition at loop exit:

$$I \wedge \neg B \Rightarrow Q.$$

Thus exactness describes the loop independently of a target, whereas Q -sufficiency is relative to one postcondition. The final verifier must prove both soundness obligations and this exit obligation.

2.2 TARGET-HIDDEN VERIFICATION

In *target-hidden loop verification*, the generator and all intermediate feedback see $\mathcal{L} = (P, B, S)$ but not Q . The postcondition is restored only after the generated invariant has been fixed. Preconditions and executable loop semantics remain visible, while assertions, postconditions, assertion calls, and conventional error-label names are masked. The untouched program is used only for final verification.

2.3 SCOPE AND MOTIVATING EXAMPLE

We focus on numerical loops for two reasons. First, their invariants are predominantly arithmetic equalities and inequalities, including polynomial relations. These predicates are comparatively amenable to automatic checking, which provides a scalable and objective verification signal. Second, numerical loops factor out the representational machinery of arrays, heaps, and recursive data structures, whose verification often requires auxiliary functions or inductive predicates. This gives a controlled setting in which success depends more directly on discovering algebraic relations and reasoning about loop dynamics.

Concretely, we study one braced `while` loop over scalar C integers, including nonlinear updates; arrays, heaps, and recursive structures are out of scope. This restriction does not make invariant discovery trivial: even a small loop can hide a nonlinear relation across coupled recurrences.

Consider the following abridged program:

```
1 int x = 1, y = 1, c = 1;
2 while (c < k) { c++; x = x*z + 1; y = y*z; }
3 /*@ assert 1+x*z-x-z*y == 0; */
```

The visible assertion can be copied verbatim as an invariant; when hidden, its nonlinear relation must instead be recovered from the coupled updates.

3 RELATED WORK

Programmatic invariant synthesis. Classical methods infer invariants with abstract interpretation, affine equalities, templates, interpolation, recurrences, or constraint solving (Karr, 1976; Cousot & Cousot, 1977; Cousot & Halbwachs, 1978; Sankaranarayanan et al., 2004; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017). These methods provide strong guarantees for selected languages, especially affine transition systems, but nonlinear templates trade generality for increasingly difficult algebraic search. More generally, proving inductiveness certifies that an invariant is a sound over-approximation of reachable loop states, but does not say how precise that over-approximation is. Many weak invariants are therefore correct, while a strongest invariant may be unavailable or inexpressible in the chosen language. Dynamic approaches infer likely relations from execution traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). LOOPGYM instead uses executions to construct a graded, target-independent training signal for precision; final soundness is still established independently by the verifier. Target-directed software model checkers constitute a related but distinct setting. For example, ULTIMATE AUTOMIZER (UAuto) uses trace abstraction and interpolation to verify a supplied safety property (Heizmann et al., 2013). Although its correctness witnesses may contain inductive assertions, they are synthesized with the verification target visible. Our setting withholds that target until the candidate set is fixed, so neither goal-directed refinement nor target counterexamples can provide the strength signal.

Neural and LLM-based invariant generation. Learned methods broaden the candidate language by using neural generators, counterexamples, or verifier feedback (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020). In particular, CODE2INV learns a sequential invariant-construction policy from sparse verifier reward (Si et al., 2018; 2020). LOOPY generates candidate clauses, filters them with Houdini, and repairs failures using verifier feedback (Kamath et al., 2023). CLAUSE2INV iteratively combines clauses using historical counterexamples and failed checks to guide later generation, while IRANK learns to rank LLM-generated invariants before expensive verifier calls (Cao et al., 2025; Chakraborty et al., 2023). Thus, clause generation, combination, Houdini filtering, and learned invariant policies are not individually new. Unlike counterexample-guided iterative repair, our inference procedure receives no feedback between rollouts: it samples a fixed group once and applies union and Houdini only after generation. The target remains hidden throughout generation and training.

Specification generation. LLM-based tools also synthesize broader program specifications. AUTOSPEC repeatedly proposes and verifies annotations; SPECGEN combines LLM proposals with mutation in Java/JML; and SESPEC uses symbolic execution to generate goal-independent specifica-

tions for C (Wen et al., 2024; Ma et al., 2025; Yang et al., 2026). We study only loop invariants and ask whether they prove a held-out target, rather than evaluating a general behavioral specification. This restriction exposes a useful compositional structure: invariant responses can be split into clauses, verified independently or jointly, and recombined without asking a model to merge natural-language rationales or rewrite a complete contract.

Reinforcement learning from formal feedback. Verifier-guided reinforcement learning has been used for program synthesis and formal reasoning. RE:FORM uses scalable curation and verifier-guided RL for Dafny, while SPECRL generates impossible input–output pairs and rewards specifications that reject them (Yan et al., 2025; Huang et al., 2026). Validity alone does not distinguish a useful specification from a weak one. SPECRL is closest to our negative-example signal: it measures the strength of each generated specification by the impossible behaviors that the specification excludes. Its negative examples are generic method-level input–output pairs: the model proposes inputs, execution supplies outputs, and output mutation constructs impossible pairs. Our sampler instead targets loop-head state space with relational perturbations, post-exit continuations, and range perturbations. Moreover, SPECRL scores a complete generated specification independently. We instead compute rewards jointly across a rollout group: one rollout’s credit depends on the verified behaviors already covered by its peers. We also use verification to retain and reward a sound clause subset inside an imperfect response, rather than assigning correctness to the response as one indivisible object. SPECRL does not split loop invariants into clauses or optimize the cross-rollout union selected by Houdini.

Verifiable-reward RL can improve small-budget accuracy while reducing the coverage available from extensive sampling (Yue et al., 2025). This tension also motivates evaluating both standalone and multi-sample coverage. Our method-level response is described in Section 4: SFT and RL are trained around the same fixed-group union-and-Houdini operator used at deployment, rather than extending a standalone verifier reward.

4 METHOD

4.1 OVERVIEW

Using the notation of Section 2, the model receives only the target-masked loop $\mathcal{L}$. We parse each response, split every invariant at its top-level conjunctions, normalize the resulting clauses, and obtain an ordered list $A = (a_1, \dots, a_m)$. We cap each response at K clauses,

$$\bar{A} = (a_1, \dots, a_{\min(m, K)}), \quad o(A) = \max(0, m - K),$$

to prevent a response from gaining coverage by producing an unbounded list.

Our *rollout–combine–filter* framework is shared by data construction and deployment. It samples responses, combines their clauses, iteratively removes syntax failures, optionally performs AST deduplication, and then applies Houdini. SFT turns the filtered combination into one target, RL improves the responses that populate a combination, and inference applies the same framework before restoring Q . This compositional operator is the central method; SFT construction, execution-derived negatives, and the RL credit rule are mechanisms for training the responses that feed it.

The reward mirrors this structure in two directions: it *decomposes within* each rollout and *composes across* rollouts. Within a response, clauses are filtered and credited separately, so an imperfect clause does not erase useful partial progress. Across a response group, coverage is combined and allocated according to what each rollout contributes beyond its peers.

4.2 ROLLOUT–COMBINE–FILTER

Rollout. The model emits ACSL (Baudin et al., 2021) annotations of the form `loop invariant <expr>;`. We split each `<expr>` at top-level logical conjunctions, so a conjunction contributes separate clauses that can survive or fail independently. This makes the clause, rather than the entire response, the unit retained for later credit. Splitting precedes capping and normalization.

```

270 Input: masked loop  $\mathcal{L}$ , responses  $A_{1:G}$ , cap  $K$ , AST-dedup flag  $d$ 
271 Output: filtered clause set  $C$ 
272 1:  $C \leftarrow \bigcup_{i=1}^G \text{Cap}_K(\text{Normalize}(\text{Split}_\wedge(A_i)))$ 
273 2: repeat
274 3:    $E \leftarrow \text{ParserErrors}(\text{Annotate}(\mathcal{L}, C))$ 
275 4:    $C \leftarrow C \setminus E$ 
276 5: until  $E = \emptyset$  or  $C = \emptyset$ 
277 6: if  $C = \emptyset$  then return  $C$ 
278 7: if  $d$  then  $C \leftarrow \text{ASTDedup}(C)$ 
279 8: repeat
280 9:    $V \leftarrow \text{WP}_{\text{est,pres}}(\mathcal{L}, C)$ 
281 10:   $D \leftarrow \{c \in C : \neg V_{\text{est}}(c) \vee \neg V_{\text{pres}}(c)\}$ 
282 11:   $C \leftarrow C \setminus D$ 
283 12: until  $D = \emptyset$  or  $C = \emptyset$ 
284 13: return  $C$ 

```

Algorithm 1: Rollout–combine–filter

Combine. Given G responses, we union their clause lists:

$$X_0 = \bigcup_{i=1}^G \bar{A}_i.$$

If I_1 and I_2 are sound invariants of the same loop, then $I_1 \wedge I_2$ is also sound and no weaker than either constituent. Combination can therefore preserve a bound from one response and a relation from another even when neither response proves the target alone. The raw union can nevertheless contain syntactically invalid or non-inductive clauses.

Filter. Algorithm 1 gives the full framework. The filter first iteratively removes clauses identified by the verifier’s parser or type checker. It may then deduplicate the syntax-valid clauses by their normalized ASTs. Finally, Houdini iteratively removes every clause whose establishment or preservation obligation fails under the verifier (Flanagan & Leino, 2001). We denote the surviving set by $\text{Filter}_{\mathcal{L}}(X)$. Filtering can retain compatible clauses from imperfect responses but cannot invent a relation absent from the combined pool. The verifier is therefore used as a projection onto a sound clause subset, not as a binary reward for the correctness of the complete response.

During RL scoring, we additionally apply a positive-example filter before the verifier pipeline. Given sampled reachable states $\mathcal{P}$, it retains only clauses consistent with every observed execution:

$$\text{PF}_{\mathcal{P}}(X) = \{a \in X : \forall p \in \mathcal{P}, p \models a\}.$$

The syntax filter and Houdini then process $\text{PF}_{\mathcal{P}}(X)$. This inexpensive step removes clauses already falsified by known reachable states; it is not a proof of soundness, which still comes from the verifier. Inference has no execution examples and omits $\text{PF}_{\mathcal{P}}$; all uses otherwise share clause splitting, capping, normalization, syntax filtering, and Houdini.

4.3 COMPRESSING MULTI-RESPONSE SEARCH INTO SFT TARGETS

For every masked loop $\mathcal{L}$, we sample a response group, combine its clauses, and compute $\text{Filter}_{\mathcal{L}}(X)$. A nonempty result is serialized as one canonical response:

$$\mathcal{D}_{\text{SFT}} = \{(\mathcal{L}, \text{serialize}(\text{Filter}_{\mathcal{L}}(X)))\}.$$

This distills a filtered multi-response result into one answer. Neither side contains Q : the input is masked and the output contains only inductive clauses. Execution states are used only for RL, never as SFT answers.

4.4 EXECUTION-BASED TRAINING EXAMPLES

To rank inductive sets without revealing Q , we derive a strength signal from executions. The sampler instruments one braced `while` loop, compiles it with `gcc`, and records loop-head valuations for a

deterministic, precondition-checked input sweep. The recorded states form $\mathcal{P} \subseteq \text{Reach}(\mathcal{L})$; clauses falsified by these states are removed before scoring. If any run hits the iteration cap, the sample is marked incomplete and cannot support terminal or range examples.

The sampler then constructs three kinds of negative examples without reading an assertion or postcondition:

- **Relational perturbations** change one or two variables by $\pm\{1, 2\}$ in densely observed states, testing relations such as $x + y = n$.
- **Post-exit continuations** execute the real loop body for additional steps after a genuine exit.
- **Range perturbations** move one variable outside its observed range and test useful bounds.

Figure 2 illustrates how the three families are generated from observed executions and then converted into negative traces.

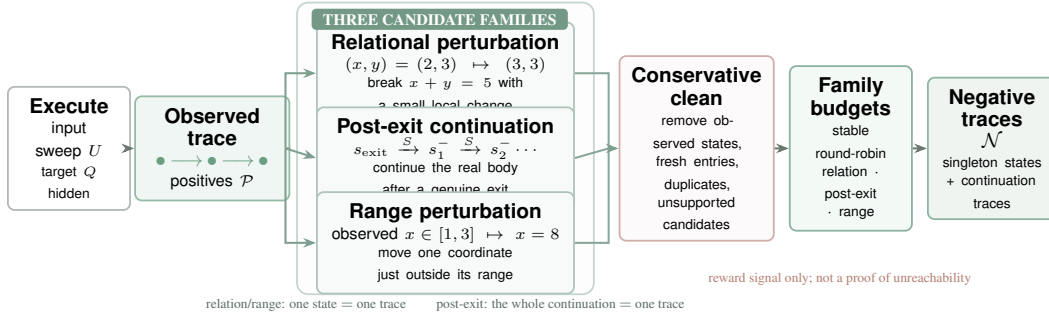


Figure 2: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Three transformations probe relational structure, behavior past a genuine exit, and sampled bounds. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q .

These are training signals, not proofs of global unreachability. We remove observed reachable states, possible fresh loop entries, duplicates, and candidates that depend on unsupported state. Relation and range candidates form singleton negative traces, whereas all states from one post-exit continuation form one trace. Each family has an independent budget, filled round-robin across variable and direction buckets. Pointers, arrays, calls, unsupported locals, or oracle-dependent behavior disable the affected construction; a program with no safe variable contributes positives only. Capped executions contribute positives but disable constructions that require complete terminal or range information. Algorithm 2 in Appendix A.1 and Table 6 give the sampling procedure and exact configuration.

Let $\mathcal{N}$ denote the retained negative traces. One trace can contain several witness states from a perturbed execution. For a candidate set X , let $\text{rej}(X) \subseteq \mathcal{N}$ be the traces on which at least one clause in $\text{Filter}_{\mathcal{L}}(X)$ is false.

4.5 CLAUSE-DECOMPOSED AND GROUP-COMPOSITIONAL REWARD

The deployed union needs strong individual responses that remain complementary within a group. Rather than treating each rollout as one indivisible object, our reward first splits it into clauses and then relates its surviving subset to the other rollouts:

$$A_i \xrightarrow{\text{Split} + \text{Filter}_{\mathcal{L}}} S_i \xrightarrow{\text{rej}} R_i, \quad (R_1, \dots, R_G) \xrightarrow{\text{union} + \text{Shapley}} (r_1, \dots, r_G),$$

where

$$S_i = \text{Filter}_{\mathcal{L}}(\text{PF}_{\mathcal{P}}(\bar{A}_i)).$$

The first arrow implements *intra-rollout decomposition*; the second stage implements *inter-rollout composition*.

Within a response, the reward measures the strength of S_i ; it does not add a response-level bonus for valid syntax or for every emitted clause surviving verification. A malformed or non-inductive

clause is removed, but does not erase credit earned by useful clauses in the same response. Across responses, the total reward of A_i depends on the rejection sets of every other response in the group. Thus decomposition preserves partial progress within a rollout, while composition assigns credit for complementary progress across rollouts.

Inductiveness supplies a binary soundness certificate, not a graded measure of precision. Every sound invariant over-approximates the reachable loop-head states, while the exact set may be unavailable or inexpressible; predicates such as *true* can therefore pass Houdini while remaining useless. Because Q is hidden during training, target sufficiency cannot rank these sound candidates. We instead use negative rejection as a dense, target-independent proxy for how tightly a candidate constrains behavior.

For response A_i , let $R_i = \text{rej}(S_i)$. Its negative rejection rate is

$$\text{coverage}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

This gives dense credit to a useful partial answer without querying the hidden target.

Coverage alone can cause every response to repeat the easiest high-reward clauses. For a group of G responses, define

$$v(S) = \frac{|\bigcup_{j \in S} R_j|}{|\mathcal{N}|}, \quad f(n) = \sum_{j=1}^G \mathbf{1}[n \in R_j].$$

Here $v(S)$ is the negative-trace coverage of a response coalition. Its closed-form Shapley allocation gives response A_i the group credit

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} \frac{1}{f(n)}. \quad (1)$$

For a trace rejected by $f(n)$ responses, each receives $1/f(n)$ credit: a unique rejection receives full credit, while redundant rejections share it. Equivalently, $1/f(n)$ is the probability that A_i is the first response to cover n under a uniformly random response ordering. The allocation therefore conserves the combined standalone coverage,

$$\sum_{i=1}^G \phi_i = \frac{|\bigcup_{i=1}^G R_i|}{|\mathcal{N}|}.$$

Unlike enumerating general response subsets, Equation 1 uses rejection frequencies already computed for coverage and adds no verifier calls. Individual coverage retains absolute response strength, while ϕ_i assigns the group’s combined behavioral coverage without double counting.

The resulting credit is group-conditioned rather than an independent verifier reward:

$$r_i = r(A_i; A_{1:G}).$$

Thus the same response receives less group credit when its rejections are already covered by its peers and more when it adds a distinct verified constraint. This rewards behavioral complementarity rather than surface-form difference.

We penalize only conservative duplicates within a response,

$$d_{\text{dup}}(A) = |\bar{A}| - |\text{dedup}(\bar{A})|.$$

The key normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations requiring stronger arithmetic assumptions. Unique zero-coverage clauses are not penalized because they may support another clause or the hidden target.

The generation reward is

$$r_{\text{gen}}(A_i; A_{1:G}) = w_c \text{coverage}(A_i) + w_g \phi_i - w_d d_{\text{dup}}(A_i) - w_o o(A_i), \quad (2)$$

with $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$. Clauses beyond K receive no coverage and incur overflow cost. Houdini removes failures before scoring, so one bad clause does not erase useful clauses in the response.

We also compute the rejection rate of $\text{Filter}_{\mathcal{L}}(X_0)$ for the combined pool. It controls group re-sampling but is not copied to every response. With no safe negatives, the implementation falls back to binary inductiveness for a nonempty response; all reported negative-coverage rewards otherwise use the two-level credit above.

Group-relative policy update. Each masked loop $\mathcal{L}$ samples a group of G token sequences $o_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$ from the current policy; parsing, splitting, and capping (§4.2) turn o_i into the scored response A_i . We normalize r_{gen} within its own sampling group to obtain a group-relative advantage,

$$\hat{A}_i = \frac{r_{\text{gen}}(A_i; A_{1:G}) - \text{mean}_{1 \leq j \leq G} r_{\text{gen}}(A_j; A_{1:G})}{\text{std}_{1 \leq j \leq G} r_{\text{gen}}(A_j; A_{1:G}) + 10^{-6}},$$

shared by every token of o_i , and update θ with the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right], \quad (3)$$

where $\rho_{i,t}(\theta) = \pi_{\theta}(o_{i,t} \mid \mathcal{L}, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid \mathcal{L}, o_{i,<t})$ and π_{ref} is the frozen reference policy.

Group-relative normalization is why the Shapley term matters for training, not only for evaluation. Coverage alone is bounded, so once sampling collapses onto one easiest clause every response in the group reports nearly the same reward, the group standard deviation shrinks, and $\hat{A}_i$ becomes an uninformative signal. Shapley allocation splits credit for shared rejections and gives full credit to a rejection supplied by only one response. It therefore breaks ties before normalization and rewards a rollout for its allocated contribution to the group’s combined coverage. This is still a target-independent behavioral surrogate rather than the Shapley value of final Q -sufficient verification.

4.6 DEPLOYMENT AND THE FIXED-BUDGET OBJECTIVE

At deployment, the model samples k masked responses, splits and normalizes their clauses, combines them, and runs the syntax and Houdini filters. No execution examples are constructed. Only then are the retained clauses and Q inserted into the untouched program for the final verifier proof.

The retained clause set and the invariant predicate produced by one inference invocation are therefore

$$X_k = \bigcup_{i=1}^k \bar{A}_i, \quad C_k = \text{Filter}_{\mathcal{L}}(X_k), \quad I_k = \bigwedge_{a \in C_k} a, \quad A_i \sim \pi(\cdot \mid \mathcal{L}),$$

rather than the best standalone response. Its final target utility is

$$U_Q(A_{1:k}) = \mathbf{1}[I_k \wedge \neg B \Rightarrow Q],$$

so the inference objective is

$$\max_{\pi} \mathbb{E}_{A_{1:k} \sim \pi(\cdot \mid \mathcal{L})} [U_Q(A_{1:k})].$$

Its empirical counterpart is `combine@k`. Unlike `pass@k`, it records success when no standalone response is sufficient but their filtered union is. SFT compresses a large candidate pool into stronger responses, while RL improves their coverage and complementarity. The goal is both to raise the `combine@k` curve and to make it converge with fewer rollouts; k_{95} measures the budget needed to reach the observed large-pool performance.

5 EXPERIMENTS

We organize the evaluation around five questions. RQ1 probes how direct `pass@k` and combined inference scale with the sampling budget. RQ2 compares `pass@1` and `combine@1` across model families. RQ3 compares the resulting system with specialized invariant-generation tools. RQ4 isolates the improvement from RL under both zero-shot and SFT initialization. RQ5 ablates the reward terms and negative-example sampler.

5.1 EXPERIMENTAL SETUP

Data sources and benchmarks. Training uses a separate pool of 37,958 target-hidden, scalar-integer, single-loop program records drawn primarily from SV-COMP and SyGuS; the 3,454 programs used to construct SFT targets are a subset of this pool. Appendix D audits overlap between this training pool and evaluation. We evaluate 832 integer C programs. From the CLAUSE2INV release, we use 316 linear programs (133 from CODE2INV, 84 from SyGuS 2019, and 99 from SV-COMP 2024) and 50 nonlinear programs (30 from LIPuS and 20 from SV-COMP 2024) (Yu et al., 2023; Cao et al., 2025). The remaining 466 programs are the integer fragment of Loopy’s original 469-program corpus; three floating-point programs are excluded because they fall outside our scalar-integer predicate language. Unsupported inputs remain failures in the common denominator. Because these collections reuse benchmarks from earlier work, 832 denotes the common file-level evaluation workload rather than 832 independently sourced problems.

Target-hidden evaluation. Before any model-facing step, we remove `assert` and `ensures` predicates, executable assertion calls, and conventional error-label names. Preconditions and executable loop semantics remain visible. After a method fixes its invariants, they are inserted into the untouched original program. A task succeeds only if the source parses, Frama-C/WP proves establishment and preservation for every retained invariant, and every restored target goal succeeds. Failures, timeouts, malformed outputs, and unsupported programs count as zero.

Verifier and inference. All final checks use Frama-C 31.0 (Gallium) (Kirchner et al., 2015), WP, and Z3 4.13.3. Neural experiments use the same target-hiding code, generation template, clause splitter, semantic deduplication, and Houdini implementation. We use each model’s serving interface and chat template. API models use the service’s default reasoning configuration; we do not override their reasoning effort at inference time. For checkpoints participating in our SFT or RL training pipeline, explicit thinking is disabled during evaluation to match the synthetic-data and training-rollout configuration. Unless an RQ varies k , generation uses temperature 1.0, top- p 1.0, at most 2,048 new tokens, and no re-roll or target-conditioned feedback. We do not set an additional top- k cutoff or repetition penalty beyond the serving-interface defaults. For the k -sweep, one batched request draws 100 responses per program, and every `pass@ $k$` and `combine@ $k$` point reuses that same saved response pool. We do not fix a per-response sampling seed; the 100 outputs are stochastic draws from that single batch.

Metrics. `Pass@ $k$` is the probability that at least one of k responses proves Q . Given c successful responses among $n = 100$, we estimate it as $\widehat{\text{pass@}k} = 1 - \binom{n-c}{k} / \binom{n}{k}$. Only responses that prove Q count as successes. `Combine@ $k$` is the fraction of programs proved by unioning the first k responses and applying Houdini. `Pass@ $k$` requires one response to contain the whole proof, whereas `combine@ $k$` can combine clauses from different responses. We also report $k_{95} = \min\{k : C_\pi(k) \geq 0.95 C_\pi(100)\}$, the smallest rollout budget that reaches 95% of `combine@100`. A smaller k_{95} means higher sampling efficiency.

Pending-result convention. Unfinished result cells below are marked *TBD* and remain empty until the corresponding common-protocol run is complete; populated cells come only from completed runs, and we do not extrapolate missing values from pilots.

5.2 RQ1: HOW DO PASS@K AND COMBINE@K SCALE?

We sample 100 target-hidden responses per task from four base checkpoints: Qwen3-4B, Qwen3-8B, Qwen3-14B, and Qwen3-30B-A3B (Yang et al., 2025). All points for one checkpoint are computed from the same saved response pool. We evaluate every recorded prefix; the complete numeric results, including k_{95} , appear in Appendix 8.

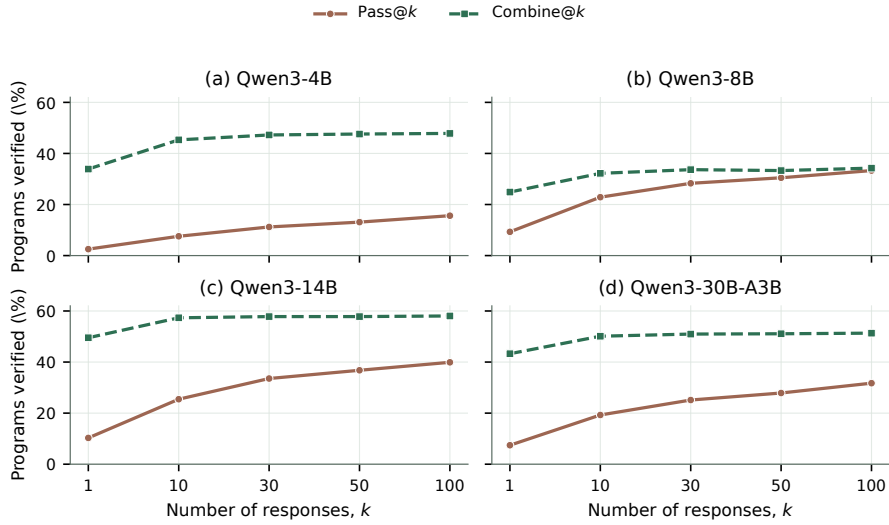


Figure 3: Scaling of $\text{pass}@k$ and $\text{combine}@k$, shown together in a separate panel for each base checkpoint. $\text{Pass}@k$ uses the unbiased estimator, whereas $\text{combine}@k$ evaluates the saved response prefix; markers denote measured budgets rather than interpolated evaluations. Combination provides a large early gain and is nearly saturated by $k = 30$, while $\text{pass}@k$ continues to grow at larger budgets.

Combination is effective and saturates early. Already at $k = 1$, $\text{combine}@1$ exceeds $\text{pass}@1$ by 31.36, 15.58, 39.23, and 35.84 percentage points for the 4B, 8B, 14B, and 30B-A3B checkpoints, respectively. This large gap shows that splitting, filtering, and recombining clauses recovers proofs even when the original response is not independently Q -sufficient. Yet almost all of this benefit is obtained quickly: every checkpoint reaches at least 95% of its $\text{combine}@100$ value by $k = 30$, and the largest additional gain from $k = 30$ to $k = 100$ is only 0.60 points. Thus, approximately 30 responses capture the practical combination ceiling in this probe.

Direct pass converges more slowly. At $k = 30$, $\text{pass}@k$ has reached only 71.8–85.0% of its $\text{pass}@100$ value across the four checkpoints. Increasing the budget from 30 to 100 still adds 4.41–6.60 percentage points, in clear contrast to the nearly flat $\text{combine}@k$ curves. Combination therefore front-loads useful proof coverage, whereas finding a single complete response continues to benefit from substantially wider sampling.

The small non-monotonicity in the 8B combine curve reflects the deployed metric: finite Frama-C/WP timeouts count as failures, and a larger clause pool can increase proof cost. We therefore interpret $\text{combine}@k$ as an operational success curve rather than assuming mathematical monotonicity.

5.3 RQ2: HOW STRONG IS COMBINE@1 ACROSS MODELS?

The main neural experiment evaluates exactly one response per program. $\text{Pass}@1$ treats that response as one all-or-nothing invariant set. $\text{Combine}@1$ splits conjunctions into clauses, removes duplicates, and runs Houdini on the resulting pool. No $\text{pass}@k > 1$ or $\text{combine}@k > 1$ result is included in this RQ, so model quality is not confounded with a larger inference budget.

management, and whether its gains remain substantial on open models. With the same `gpt-5-nano` backbone, LOOPGYM improves the naive baseline from 45.55% to 54.33% at `pass@1` and to 60.58% at `combine@1`.

5.5 RQ4: HOW MUCH DOES REINFORCEMENT LEARNING IMPROVE INFERENCE?

We use Qwen3-8B and separate two paired training paths. The zero-shot path compares the base checkpoint with RL-Zero. The supervised path compares the SFT checkpoint immediately before RL with the checkpoint obtained by continuing from it under the same RL objective. Each after-RL model therefore has a matched before-RL initialization, preventing SFT gains from being misattributed to RL.

Initialization	Training stage	pass@1	combine@1	combine@10	combine@100	k_{95}	Verified / 832
Base 8B	before RL (Zero)	TBD	TBD	TBD	TBD	TBD	TBD
Base 8B	after RL (RL-Zero)	TBD	TBD	TBD	TBD	TBD	TBD
SFT 8B	before RL (SFT)	TBD	TBD	TBD	TBD	TBD	TBD
SFT 8B	after RL (SFT+RL)	TBD	TBD	TBD	TBD	TBD	TBD

Table 3: Paired evaluation of RL from zero-shot and SFT initialization. All four rows use identical rollout budgets and the same inference code.

RL is useful only if the after-RL checkpoint improves over its matched before-RL checkpoint. We report `pass@1` as a diagnostic, but the primary outcomes are `combine@k` and k_{95} : the objective is designed to make a small group contain stronger and more complementary clauses, not merely to increase standalone proofs.

5.6 RQ5: WHAT DRIVES THE TRAINING GAIN?

Both ablation studies use Qwen3-8B and hold prompts, data order, rollout-group size, update budget, seeds, and inference code fixed.

Reward-function ablation. We separate the two axes of our credit assignment from the density of the strength signal. The conventional baseline assigns a binary verifier reward to a complete response. The second row replaces this binary outcome with negative coverage, but remains all-or-nothing: it receives coverage only when every admitted clause survives the positive-example and Houdini filters,

$$r_i^{\text{whole}} = \mathbf{1}[S_i = \bar{A}_i] \frac{|R_i|}{|\mathcal{N}|} - w_o o(A_i).$$

The third row computes the same coverage on the surviving clause subset S_i , isolating *intra-rollout decomposition*. The fourth row adds Shapley allocation across $R_{1:G}$, isolating *inter-rollout composition*. The final row adds only the duplicate regularizer. The response cap and overflow cost are held fixed across all four coverage-based rows. Thus each adjacent comparison introduces one design decision. The rows select `reward_variant=``binary`, `whole_coverage`, `base`, `base_shapley`, and `full`, respectively. We call these variants *Binary Verification*, *Whole-Rollout Strength*, *Clause-Decomposed Strength*, *Compositional Credit*, and *Regularized Compositional Credit*.

Variant	Within-rollout unit	Across-rollout credit	pass@1	combine@1	combine@5	combine@10	combine@100	k_{95}
Binary Verification	Complete response	Independent	TBD	TBD	TBD	TBD	TBD	TBD
Whole-Rollout Strength	Complete response	Independent	TBD	TBD	TBD	TBD	TBD	TBD
Clause-Decomposed Strength	Sound subset S_i	Independent	TBD	TBD	TBD	TBD	TBD	TBD
Compositional Credit (ours)	Sound subset S_i	Shapley	TBD	TBD	TBD	TBD	TBD	TBD
Regularized Compositional Credit	Sound subset S_i	Shapley + redundancy	TBD	TBD	TBD	TBD	TBD	TBD

Table 4: Credit-assignment ablation on Qwen3-8B. Adjacent rows isolate dense strength, intra-rollout decomposition, inter-rollout composition, and structural regularization, respectively.

Negative-sampling ablation. We compare only the two externally exposed samplers. Both use the same executions, reachable bases, safety filters, seed, and 512-trace total cap. *Random* perturbs one or two variables with uniformly sampled local offsets. *Structured* combines relational perturbations, post-exit continuations, and range violations under fixed family budgets. This cap-matched comparison tests whether loop-specific structure matters beyond the number of sampled negatives. The rows select `negative_sampler=random` and `structured`.

Negative sampler	Reward variance	Hard-collision rate	pass@1	combine@1	combine@10	Verified / 832
Random perturbation (same trace budget)	TBD	TBD	TBD	TBD	TBD	TBD
Structured sampler (ours)	TBD	TBD	TBD	TBD	TBD	TBD

Table 5: Negative-example sampling ablation under fixed execution and sampler caps.

Besides final verification, we report reward variance and hard-collision audits. Reward variance tests whether a sampler supplies enough discrimination for group-relative RL. The collision audit checks whether a generated negative is also observed as reachable under expanded execution sampling, guarding against gains obtained from systematically incorrect negative labels.

6 CONCLUSION AND LIMITATIONS

The central property exploited by LOOPGYM is that verified invariant clauses compose across responses. We therefore make rollout–combine–filter the shared object of training and inference, and optimize its fixed-budget `combine@k` outcome rather than `pass@1`. SFT distills a verified combination into one response; target-independent RL decomposes credit over the sound clause subset within each response and composes credit across responses based on behavior not covered by peers. Consequently, ten trained-policy responses outperform one hundred base-model responses even as `pass@1` decreases.

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require substantially richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. In particular, our state perturbations cannot yet construct reliably informative negative examples for inductive predicates or interacting nested loops. Extending both the data and negative-example semantics to these settings remains future work (Liu et al., 2024).

Inference engine. In preliminary comparisons, we observed lower verification rates when serving the locally evaluated checkpoints with vLLM than with alternative inference backends, even under nominally matched decoding settings. We nevertheless use vLLM for all locally served base and trained checkpoints: its throughput makes the sampling study practical, and using the same engine for training rollouts and evaluation avoids an additional train–test mismatch. API models retain their service defaults. The reported absolute success rates for local checkpoints may therefore be conservative and should not be read as an engine-independent ceiling.

Verifier and specification language. All filtering and final judgments use Frama-C/WP, and candidates are expressed in ACSL. Parser behavior, generated proof obligations, prover integration, and annotation syntax all affect which clauses survive the pipeline. The rollout–combine–filter principle is not intrinsically tied to this toolchain, but the quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

Feedback regime. We focus on the feedback-scarce setting and do not evaluate iterative optimization from verifier feedback. In our pilot runs, a model usually reached *sound but weak* invariants quickly: they passed inductiveness checks, but the verifier provided no signal about the missing behavioral strength while the target remained hidden. Consequently, feedback-based refinement produced little marginal gain, and that gain decreased further as the number of initial rollouts grew because their union already captured most easy corrections. Our results therefore characterize target-independent

training and finite-budget combination, not settings where a verifier exposes rich counterexamples or target-directed repair signals.

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. <https://frama-c.com/html/acsl.html>, 2021.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 84–96, 1978.
- Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 443–456, 2013.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.
- Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.
- Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 281–292, 2008.
- Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 634–640. Springer, 2009.

-
- Matthias Heizmann, Jochen Hoenicke, and Andreas Podelski. Software model checking for people who love automata. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pp. 36–52. Springer, 2013. doi: 10.1007/978-3-642-39799-8_2.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.
- Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.
- ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 318–329, 2004.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

864 Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi,
865 Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu.
866 Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A
867 preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
868
869 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang
870 Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*,
871 2025.
872
873 Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin
874 Gu. Integrating symbolic execution with LLMs for automated generation of program specifications.
875 *arXiv preprint arXiv:2506.09550*, 2026.
876
877 Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop
878 invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming
879 Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
880
881 Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced
882 reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis
883 (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.
884
885 Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang.
886 Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?
887 In *Advances in Neural Information Processing Systems*, volume 38, 2025.
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917

A IMPLEMENTATION DETAILS

Table 6 lists the implemented defaults. Execution sampling is used only to compute reinforcement-learning rewards; it does not construct the synthetic SFT corpus. SFT synthesis, RL, and inference share the rollout–combine–filter implementation: the same conjunction splitting, response cap, normalization, candidate combination, iterative syntax filter, optional AST deduplication, and Houdini pruning. SFT serializes the filtered combination, RL scores the responses that populate it, and inference restores Q only after filtering. Inference does not construct execution examples. The operators are shared, but RL applies Houdini once per response for credit assignment whereas SFT and inference apply it after union; Appendix G analyzes this ordering mismatch.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
range-perturbation steps	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	320 (≤ 64 fallback) / 64 / 128	relational / post-exit / range examples; stable round-robin within each kind
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	(1.0, 0.3)	negative rejection / Shapley group-credit weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generated response
w_o	0.05	overflow cost per clause after K
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget

Table 6: Defaults for execution sampling (§4.4), reward computation (§4.5), inference, and verification.

The exact near-input tier order is 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9.

A.1 NEGATIVE-TRACE SAMPLING

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\text{Perturb}_{1,2}$ applies the configured single-variable and pairwise changes, while RangeEscape moves one variable just outside its sampled range. The exact perturbations and budgets are those in Table 6.

```

972   Input: masked loop  $\mathcal{L} = (P, B, S)$ , input schedule  $U$ , configuration  $\Theta$ 
973   Output: reachable states  $\mathcal{P}$ , negative traces  $\mathcal{N}$ 
974   1:  $\mathcal{T}, \mathcal{C}_{\text{post}}, c \leftarrow \text{Trace}(\mathcal{L}, U, \Theta) \triangleright c: \text{some run was capped}$ 
975   2:  $\mathcal{P} \leftarrow \text{UniqueHeads}(\mathcal{T}); \quad M \leftarrow \text{SafeModifiedVars}(S)$ 
976   3: if  $M = \emptyset$  or  $S$  has unsupported state then return  $(\mathcal{P}, \emptyset)$ 
977   4:  $\mathcal{S} \leftarrow \text{StratifiedSample}(\mathcal{P})$ 
978   5:  $C_{\text{rel}} \leftarrow \text{Perturb}_{1,2}(\text{Dense}(\mathcal{S}), M, \Theta)$ 
979   6: if  $\neg c$  then  $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_{1,2}(\text{Terminals}(\mathcal{T}), M, \Theta)$ 
980   7:  $C_{\text{range}} \leftarrow \text{RangeEscape}(\mathcal{S}, M, \mathcal{P}, \Theta)$  if  $\neg c$ , else  $\emptyset$ 
981   8:  $(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}) \leftarrow \text{Clean}_{\mathcal{P}}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}})$ 
982   9:  $N_{\text{rel}}, N_{\text{post}}, N_{\text{range}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}, \Theta)$ 
983   10:  $\mathcal{N} \leftarrow \{\langle s \rangle : s \in N_{\text{rel}}\} \cup \mathcal{N}_{\text{post}} \cup \{\langle s \rangle : s \in N_{\text{range}}\}$ 
984   11: return  $(\mathcal{P}, \mathcal{N})$ 

```

Algorithm 2: Target-hidden negative-trace sampling

$\text{Clean}_{\mathcal{P}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates, using family priority relation, post-exit, then range. A cleaned post-exit continuation remains one trace; relation and range candidates become singleton traces. Within the relation budget, candidates that preserve the guard truth value and remain inside the sampled coordinate ranges are selected first.

B GENERATION PROMPT

The same generation template proposes the responses later combined during SFT synthesis, RL, and inference. Any assertion is stripped before the program is inserted. General annotation rules are supplied separately by `system_prompt.txt`.

Generation prompt

```

You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}

```

C TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and Z3. It exposes `POST /reward`, accepting program source, an array of model responses, `reward_variant` in `{binary, whole_coverage, base, base_shapley, full}`, and `sampler.negative_sampler` in `{random, structured}`. The two switches are independent; the reward defaults to `full` and the sampler to `structured`. The response exposes the selected modes and Shapley credit explicitly:

```

{program, n_positives, n_negatives, filter_mode,
 reward_variant, negative_sampler, reward_mode,
 rollout_rewards[], base[], shapley_credit[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score, should_reroll}

```

Responses may be raw LLM text, invariant lists, or annotated code; parsing is uniform. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

D TRAINING–EVALUATION OVERLAP AUDIT

The 37,958 released training records contain 36,301 unique source strings after whitespace normalization and include every program used for SFT. We compare this pool against all 832 evaluation files after removing Q , exactly as it is removed before model inference.

D.1 LAYERED MATCHING PROTOCOL

A single exact-string check can miss renamed or lightly rewritten copies. We therefore apply four increasingly aggressive program-level normalizations, all preserving `requires` clauses.

1. **Target-hidden source** normalizes whitespace after removing Q .
2. **Near-exact tokens** additionally remove non-contract comments, anonymize the loop function name, and tokenize the remaining source.
3. **Alpha-normalized source** renames identifiers in first-use order and maps the benchmark nondeterministic-function names to one token, while preserving constants, operators, statement order, and nesting.
4. **Constant-abstracted source** retains 0 and 1 but maps every other integer literal to one token. This catches copies changed only by variable names and nontrivial numeric constants.

We separately measure control-flow similarity. The *coarse profile* records the loop-guard category, capped numbers of `if/else` nodes, compound guards, and the presence of `break`, `continue`, `return`, `goto`, and nested loops. The *ordered control-flow skeleton* retains the order and brace nesting of control nodes and replaces ordinary statements by a common token. Its stricter variant also retains comparison and logical operators in branch conditions. These control-flow signatures intentionally discard data flow and arithmetic and are similarity diagnostics, not program-identity tests.

D.2 OVERLAP RESULTS

Representation	Shared signatures	Matched evaluation files
Target-hidden source	0	0 (0.0%)
Near-exact token skeleton	0	0 (0.0%)
Alpha-normalized source	0	0 (0.0%)
Constant-abstracted alpha source	0	0 (0.0%)
Coarse control-flow profile	19	764 (91.8%)
Ordered control-flow skeleton	21	522 (62.7%)
Skeleton + condition-operator shape	22	421 (50.6%)

Table 7: Layered training–evaluation overlap audit. Program-level representations preserve data flow; control-flow-only representations deliberately omit it.

No evaluation program matches the training pool at any program-level stage. In particular, overlap remains zero after all identifiers are alpha-renamed and after nontrivial constants are abstracted. This rules out exact copies, function-name or formatting variants, variable renamings, and copies changed only by numeric constants under the stated normalizations.

Control flow alone overlaps substantially. The coarse profiles of 764 of 832 evaluation files occur in training; preserving statement order and nesting reduces this to 522 files, and retaining condition-operator shapes reduces it to 421. This is expected for a scalar single-loop language: straight-line loops, one-branch loops, and `if--else` updates form a small reusable vocabulary. The contrast is the relevant observation: training covers common control-flow forms, but every match disappears once arithmetic and data-flow structure are retained. The observed control-flow overlap is therefore consistent with task coverage and does not by itself establish memorization.

Scope and residual risk. This audit covers the released fine-tuning data, not unknown base-model pretraining corpora. It may also miss algebraically equivalent updates, reordered independent

statements, transformed loops, or distant semantic clones. We therefore report no overlap under the stated normalizations rather than claiming complete semantic independence.

E COMPLETE EXPERIMENTAL DATA

This section reports the complete experimental-data schema used for the final paper. We include per-model, per-suite, and per-seed fields so that aggregate numbers in the main text can be traced to their underlying measurements. Entries marked TBD will be filled from the final evaluation artifacts; no missing value is used in the analysis.

E.1 PROBE CURVES

Table 8: Complete probe results. k_{95} is the smallest k at which $\text{combine}@k$ reaches 95% of its value at $k = 100$.

Model	pass@1	pass@10	pass@30	pass@50	pass@100	combine@1	combine@10	combine@30	combine@50	combine@75	combine@100	k_{95}
Qwen3-4B	2.53%	7.56%	11.21%	13.10%	15.62%	33.89%	45.31%	47.24%	47.60%	47.72%	47.84%	30
Qwen3-8B	9.30%	22.87%	28.30%	30.47%	33.29%	24.88%	32.21%	33.65%	33.29%	32.93%	34.25%	30
Qwen3-14B	10.29%	25.45%	33.55%	36.78%	39.90%	49.52%	57.33%	57.81%	57.81%	57.93%	58.05%	10
Qwen3-30B-A3B	7.43%	19.27%	25.13%	27.88%	31.73%	43.27%	50.12%	50.96%	51.08%	51.20%	51.32%	10

E.2 REINFORCEMENT-LEARNING RESULTS

Table 9: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Path	Model	pass@1	combine@1	combine@5	combine@10	combine@50	combine@100	k_{95}
Zero	Base 8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Zero	Base 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT	SFT 8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT	SFT 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD

E.3 ABLATION RESULTS

Table 10: Complete reward-ablation results on the 8B model. Reported values are verification rates; $\text{Verified} \in \{0, 1\}$ records whether the final union is accepted.

Reward	Seed 1	Seed 2	Seed 3	Mean	Std.	Verified
Binary Verification	TBD	TBD	TBD	TBD	TBD	TBD
Whole-Rollout Strength	TBD	TBD	TBD	TBD	TBD	TBD
Clause-Decomposed Strength	TBD	TBD	TBD	TBD	TBD	TBD
Compositional Credit	TBD	TBD	TBD	TBD	TBD	TBD
Regularized Compositional Credit	TBD	TBD	TBD	TBD	TBD	TBD

Table 11: Complete negative-trace sampler ablation on the 8B model.

Sampler	Seed 1	Seed 2	Seed 3	Mean	Std.	Verified
Random perturbation	TBD	TBD	TBD	TBD	TBD	TBD
Structured sampler	TBD	TBD	TBD	TBD	TBD	TBD

E.4 CROSS-MODEL MAIN RESULTS

Table 12: Complete main results. Each benchmark column reports pass@1/combine@1.

Model	Code2Inv	SyGuS	SV-COMP	LIPuS	Loopy	Overall	Time	Output tokens
GPT-5-nano	TBD	TBD	TBD	TBD	252/294 (54.08/63.09%)	TBD	TBD	TBD
GPT-5-mini	TBD	TBD	TBD	TBD	284/353 (60.94/75.75%)	TBD	TBD	TBD
GPT-5	TBD	TBD	TBD	TBD	375/375 (80.47/80.47%)	TBD	TBD	TBD
Claude Sonnet-4.6	TBD	TBD	TBD	TBD	TBD	56.13%/63.46%	59.88 s [†]	1,193.55 [†]
DeepSeek-V4-Flash	TBD	TBD	TBD	TBD	243/254 (52.15/54.51%)	TBD	TBD	TBD
Qwen3-14B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama (checkpoint TBD)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

E.5 COMPARISON WITH SPECIALIZED TOOLS

Table 13: Complete target-hidden tool-comparison results. Each cell reports verified programs and rate under the common file-level denominator.

Method	Linear / 316	NLA / 50	Loopy / 466	Overall / 832	Tokens / task	Time / task
AutoSpec	208 (65.82%)	4 (8.00%)	302 (64.81%)	514 (61.78%)	25,154.54	54.77 s
SESpec	180 (56.96%)	3 (6.00%)	231 (49.57%)	414 (49.76%)	44,807.05	117.31 s
Clause2Inv	63 (19.94%)	1 (2.00%)	0 (0.00%)	64 (7.69%)	385.80	29.04 s
Daikon	73 (23.10%)	0 (0.00%)	101 (21.67%)	174 (20.91%)	0	4.89 s
Naive	154 (48.73%)	4 (8.00%)	221 (47.42%)	379 (45.55%)	4,493.37	28.87 s
LoopGym (pass@1)	193 (61.08%)	7 (14.00%)	252 (54.08%)	452 (54.33%)	6,928.30	53.52 s
LoopGym (combine@1)	203 (64.24%)	7 (14.00%)	294 (63.09%)	504 (60.58%)	6,928.30	66.60 s
LoopGym (combine@5)	250 (79.11%)	7 (14.00%)	365 (78.33%)	622 (74.76%)	33,404.56	284.81 s
LoopGym (combine@10)	255 (80.70%)	7 (14.00%)	376 (80.69%)	638 (76.68%)	68,953.35	536.81 s

F WHY THINKING IS DISABLED FOR TRAINED MODELS

API models use their default reasoning configuration. For checkpoints trained in our pipeline, however, we disable chain-of-thought (CoT) during synthetic data construction, training rollouts, and evaluation inference. Applying the same setting at all three stages avoids a train–test distribution shift. This choice has two practical motivations.

CoT substantially increases generation cost. Our inference procedure samples many rollouts for each program. Explicit CoT adds reasoning tokens and generation latency to every rollout, so its cost is multiplied by the sampling budget. The required output, however, is only a short, machine-parseable set of ACSL clauses. We therefore omit explicit CoT to keep both data generation and multi-rollout inference efficient.

Invariant clauses compose, but CoTs do not. A single rollout may contain both valid and invalid candidate invariants. The invariant output is explicitly decomposable into clauses: Houdini can remove a failing clause while retaining the useful clauses, and the retained clauses can then be combined across rollouts. A CoT does not have this property. Correct and incorrect reasoning may be interleaved in one trace, with no reliable clause-level correspondence that would let us remove the reasoning associated with a rejected clause. Nor is there a principled way to merge the remaining fragments from different rollouts into one faithful CoT for the combined invariant set. Generating a new rationale after filtering would only produce a post-hoc explanation. Consequently, CoT supervision is not aligned with the clause-wise filter-and-combine target. For consistency, CoT remains disabled for our trained checkpoints during synthetic-data construction, RL training, and inference.

G TRAINING–INFERENCE MISMATCH IN HOUDINI ORDERING

Let A_i be the invariant set proposed by rollout i , and let $H(X)$ denote syntax filtering followed by Houdini on candidate set X . SFT construction and inference first merge the candidates and then filter them:

$$S_{\text{infer}} = H\left(\bigcup_{i=1}^G A_i\right). \quad (4)$$

This order allows a clause from one response to be preserved with the help of a companion clause proposed by another response.

RL requires a scalar reward for each rollout. The current implementation therefore runs Houdini independently and computes coverage, Shapley credit, and redundancy from

$$S_i^{\text{train}} = H(A_i). \quad (5)$$

It also evaluates $H(\bigcup_i A_i)$ as a group-level diagnostic and for reroll decisions, but this shared result does not directly determine each rollout’s token-level advantage.

Why the two orders are not equivalent. Houdini is not distributive over union:

$$H\left(\bigcup_i A_i\right) \neq \bigcup_i H(A_i) \quad \text{in general.} \quad (6)$$

For example, a clause from one rollout may fail preservation in isolation but become inductive when conjoined with a supporting clause from another rollout. Thus, two clauses rejected by separate Houdini runs can belong to an inductive conjunction after union. Because the training reward filters each response first, it misses the positive credit associated with this cross-rollout support.

Why the practical effect is small. This is missed positive credit, not an explicit penalty for non-inductiveness. The reward assigns coverage and Shapley credit to surviving clauses and penalizes duplicates and overflow, but it does not penalize a unique clause merely because it fails the standalone Houdini check. Such a clause therefore receives no inductiveness-derived reward, yet the policy is not directly trained to suppress it. At inference time, raw candidates from all responses are unioned before Houdini, so a complementary pair remains available and can be retained jointly. The mismatch therefore does not change the inference procedure or remove cross-response support from its candidate pool. In our experiments, cases rescued only by cross-rollout inductiveness occur with low probability, making this a small mismatch in practice.

Why retain this approximation. Independent filtering gives a stable, parallelizable reward for every response. Assigning the same union-level score to all G responses is uninformative under group-relative normalization because the shared component cancels. Exact leave-one-out credit,

$$\Delta_i = V\left(H\left(\bigcup_j A_j\right)\right) - V\left(H\left(\bigcup_{j \neq i} A_j\right)\right),$$

requires at least $G + 1$ union-level Houdini evaluations per prompt and can still miss higher-order interactions. Shapley-style credit captures such interactions more faithfully but requires many subset evaluations. This would be the Shapley value of the union-and-Houdini verifier game, unlike the closed-form Shapley allocation of negative-set coverage used by our reward.

Implications and future correction. The ordering difference makes standalone Houdini a slightly imperfect reward surrogate for $\text{combine@}k$, but it does not alter inference-time union-and-Houdini. Future work can report the rescue rate represented by the gap between the two sides of Equation 6 and test leave-one-out or randomized-subset credit when that rate is non-negligible. For the present experiments, the observed rarity of these rescues does not justify the additional verifier cost.